LRU Cache Implementation - Eviction Policy

Theory, Edge Cases & Code Examples

- ✓ LRU Cache Concept
- ✓ Data Structure Design
- ✓ Accessing Non-Existent Key
- ✓ Capacity = 1
- ✓ Page Cache

→ Swipe to learn →

LRU Cache Implementation - Eviction Policy

Theory

📌 1. LRU Cache Concept

Goal: Fixed-size cache that evicts least recently used items when full.

Operations:
- `get(key)`: Return value, mark as recently used
- `put(key, value)`: Insert, evict LRU if full

Use cases:
- Web browser cache
- Database query cache
- Page replacement in OS

LRU Cache Implementation - Eviction Policy

Theory

📌 2. Data Structure Design (1/2)

Requirements:
- O(1) get
- O(1) put
- Track access order

Solution: Hash map + doubly-linked list

```
HashMap: key → list_node*

List (MRU → LRU):
[most recent] ↔ [item2] ↔ [item3] ↔ [least recent]
```

```
#include <unordered_map>
#include <list>
#include <optional>

template<typename Key, typename Value>
class LRUCache {
private:
    struct Node {
        Key key;
        Value value;
    };

    size_t capacity_;
    std::list<Node> items_;  // MRU at front
    std::unordered_map<Key, typename std::list<Node>::iterator> map_;

public:
    explicit LRUCache(size_t capacity) : capacity_(capacity) {}

    std::optional<Value> get(const Key& key) {
        auto it = map_.find(key);

        if (it == map_.end()) {
            return std::nullopt;
        }

// ...continues on next slide
```

LRU Cache Implementation - Eviction Policy

Theory

📌 2. Data Structure Design (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        // Move to front (most recently used)
        items_.splice(items_.begin(), items_, it->second);

        return it->second->value;
    }

    void put(const Key& key, const Value& value) {
        auto it = map_.find(key);

        if (it != map_.end()) {
            // Update existing
            it->second->value = value;
            items_.splice(items_.begin(), items_, it->second);
            return;
        }

        // Insert new
        if (items_.size() >= capacity_) {
            // Evict LRU (back of list)
            auto lru = items_.back();
            map_.erase(lru.key);
            items_.pop_back();
        }

        items_.push_front({key, value});
        map_[key] = items_.begin();
    }

    size_t size() const {
        return items_.size();
    }
};
```

LRU Cache Implementation - Eviction Policy

Edge Cases

📌 1. Accessing Non-Existent Key

```
auto val = cache.get(missing_key);
if (val) {
    // Use *val
} else {
    // Handle miss
}
```

LRU Cache Implementation - Eviction Policy

Edge Cases

📌 2. Capacity = 1

```
LRUCache<int, int> cache(1);
cache.put(1, 10);
cache.put(2, 20);  // Evicts key 1
auto val = cache.get(1);  // nullopt
```

LRU Cache Implementation - Eviction Policy

Code Examples

📌 1. Page Cache (1/2)

This example demonstrates **an LRU cache simulating a web browser page cache**, where recently-accessed pages remain in memory for instant retrieval, while least-recently-used pages are evicted when the cache fills. The code shows the classic cache hit/miss pattern and automatic eviction behavior when capacity is exceeded.

What this code does:

- Creates an LRU cache with capacity 3, storing page numbers (int) → page content (string)
- `access_page()` function first checks if page is cached using `get()` (cache hit)
- On cache miss, simulates loading page from disk/network and inserts into cache using `put()`
- Sequence: Access pages 1,2,3 (all misses, cache now full), then page 1 (hit - moves to front)
- Accessing page 4 exceeds capacity → evicts page 2 (least recently used: hasn't been accessed since initial load)
- Accessing page 2 again results in miss because it was evicted (demonstrates LRU eviction policy)

Key concepts demonstrated:

- **Cache hit optimization** - page 1 access after being cached returns instantly from memory (no disk I/O)
- **Automatic eviction** - cache manages its own size, evicting LRU items when capacity reached
- **Access-order tracking** - each `get()` moves item to front of list (marks as most-recently-used)
- **LRU policy in action** - page 2 evicted (not accessed recently) even though it was one of the first loaded
- **std::optional usage** - `get()` returns optional to safely handle cache misses without exceptions

Why this matters:

Modern software deals with vast amounts of data that can't all fit in memory. **LRU caching provides O(1) access to frequently-used data while automatically managing limited memory**. Web browsers use this exact pattern: keeping recently-viewed pages in RAM for instant back/forward navigation, evicting old pages when memory fills. Without LRU, every page access would require disk/network I/O (1000× slower than RAM).

Performance implications:

- Cache hit: ~1ns hash lookup + ~1ns list splice = instant access (no disk I/O at ~1-10ms)
- Cache miss: O(1) insertion with possible O(1) eviction, then expensive load operation
- Memory bounded: 3-page cache uses ~constant memory regardless of total pages accessed
- Hit rate matters: 90% hit rate = 10× average speedup; 50% hit rate = 2× speedup
- Capacity tuning: larger cache = higher hit rate but more memory; smaller = more evictions

Real-world applications:

- **Web browsers**: Page cache (Firefox/Chrome keep ~100-500 pages in memory)
- **Operating systems**: Page table cache, disk block cache (buffer cache in Linux)
- **Databases**: Query result cache, index page cache (PostgreSQL shared buffers, MySQL query cache)
- **CDNs**: Content delivery edge servers cache popular files (Cloudflare/Akamai)
- **API servers**: Rate limiting, session storage, frequently-accessed database rows

Output:

LRU Cache Implementation - Eviction Policy

Code Examples

📌 1. Page Cache (2/2)

```
#include <iostream>
#include <string>

LRUCache<int, std::string> page_cache(3);

void access_page(int page_num, const std::string& content) {
    if (auto cached = page_cache.get(page_num)) {
        std::cout << "Cache HIT: Page " << page_num << '\n';
    } else {
        std::cout << "Cache MISS: Page " << page_num << ", loading...\n";
        page_cache.put(page_num, content);
    }
}

int main() {
    access_page(1, "Page 1 content");  // MISS
    access_page(2, "Page 2 content");  // MISS
    access_page(3, "Page 3 content");  // MISS
    access_page(1, "");  // HIT
    access_page(4, "Page 4 content");  // MISS (evicts page 2)
    access_page(2, "");  // MISS (page 2 was evicted)

    return 0;
}
```

```
['Cache MISS: Page 1, loading...\nCache MISS: Page 2, loading...\nCache MISS: Page 3, loading...\nCache HIT: Page 1\nCa
```

What You Learned

Key Takeaways

- ✓ LRU Cache Concept
- ✓ Data Structure Design
- ✓ Accessing Non-Existent Key
- ✓ Capacity = 1

→ Swipe to learn →